

项目管理看板 · 产品手册

多项目本地看板 · 全景参考（给人与模型）

生成日期 2026-07-02

项目管理看板 · 全景产品手册（Product Manual）

本手册的读者是「没法读源码、也不了解本项目背景」的人或 AI 模型。目标：读完这一份，你就能准确回答关于本产品的定位、能力、架构、数据模型、设计取舍、已知边界与使用方式的问题，无需再翻代码。版本对应：产品 v1.0.0（社区分发版）。文中路径、命令、字段名均与实现一致。

0. 一句话与电梯陈述

- 一句话：**一个本地运行的、多项目通用的项目管理可视化看板——把「每个项目里每件事做到哪一步、卡在哪、有什么等人拍板」一屏看清，并能**直接在网页上拍板**。
- 电梯陈述：**它是一层**读多写少**的项目治理面板。真正的「事实」存在一个机器可读的 `board.json` 里；一个零依赖 CLI 是**唯一的写入通道**；一个零依赖本地 server 提供 API + 实时推送；一个 Vue 前端提供 13 个视图。它最初为「AI (Claude Code) 施工、人类监督拍板」的协作模式而生，也可作为通用的轻量项目面板独立使用。

1. 产品定位

1.1 它是什么

一个**单机、离线、零云端**的项目管理面板。安装后在本机 <http://127.0.0.1:6060/> 打开网页，展示一个或多个项目的任务状态、进度、风险、依赖、待决策事项，并支持在网页上对「待拍板」的问题一键拍板。

1.2 为谁做的

- 核心用户：**用 AI / 大模型 (Claude Code 等) 驱动开发的个人或小分队。任务是「派」给模型的：模型接单 → 建立完整任务链 → 经 CLI 把状态/进度/决策/落地全程写回看板。人类不手动维护任务，只负责「看进度 + 岔路口拍板 + 派活给模型」。
- 这正是它区别于传统敏捷看板 (Jira/Trello) 的本质：**传统看板靠人手动挪卡、改状态；本产品靠**模型自动更新**，人是监督者与决策者，不是数据录入员。
- 不适合：**想要「纯手工维护、人来挪卡改状态」的传统 PM 用法——那样它就退化成一个更弱的传统看板，用错了地方（见 §7 核心哲学与 §10 边界）。

1.3 解决什么痛点

- 多项目状态分散：**几个项目的进度散落在各处，没有统一的一屏总览。

- 2. **AI 施工的进度不透明**：AI 在多个对话里干活，人类不知道每件事做到哪、卡在哪。
- 3. **决策断链**：AI 遇到岔路口需要人拍板，但「问题→答案→谁去落地→是否落地了」经常断链、丢失。
- 4. **数据新鲜度不靠谱**：靠人/对话「记得」去更新状态，必然漂移。本产品用 git hook + 读时派生把机械字段自动化。

1.4 它不是什么（定位边界）

- **不是 SaaS / 云看板**（如 Jira、Trello、Linear）。它是单机本地工具，无账号、无服务器、无协作实时同步、默认只绑 127.0.0.1 本机访问。
- **不是通用团队协作工具**。没有多人权限、评论、通知推送、移动端。
- **不是「拖拽式」人工项目管理器**。**看板的写入由施工方（AI/模型）经 CLI 完成，人类不手动挪卡改状态**；网页端给人的写操作只有「拍板」。这是刻意设计（见 §7 核心哲学），不是功能没做完。
- **不是跨平台成品分发**。当前社区分发版是 **Windows NSIS 安装器**；源码可在 macOS/Linux 跑（有 dashboard.sh），但未打包。

2. 核心概念（术语表）

术语	含义
项目（project）	一个被看板管理的代码仓 / 工作目录。每个项目有一个 id、name、mainRepo（主仓路径）。
registry（注册表）	registry.json，记录「有哪些项目、各自的路径」。是项目列表的唯一来源。
board (board.json)	单个项目的 全部任务数据 。每个项目一份，落在该项目主仓的 <mainRepo>/.dashboard/board.json。 不进 git 。
任务（task）	项目里的一件事（一个功能、一个 bug、一个决策批次）。有 id、标题、状态、进度、分支、决策、依赖、文档等字段。
状态（status）	任务所处阶段，10 种枚举（见 §5.2）。
决策 / 拍板 (decision)	挂在某任务上的「需要人拿主意」的问题。有问题、选项、推荐项、答案。 拍板 = 填上答案。
活动流（activity）	每次写操作追加一条流水（谁、何时、做了什么）。
DASHBOARD_HOME	「数据根」目录：registry.json 与 snapshots/ 落此处。默认 ~/.claude/dashboard；分发版指向安装目录（见 §4.4）。
波次（wave）	任务所属的「第几批做」的计划批次编号（整数，默认 0）。
落地（landed）	一条决策被拍板后，是否已经有人真正把它实现到代码里。

3. 用户可见特性

3.1 顶栏

- **项目下拉框**：切换当前查看的项目。

- **连接状态圆点**：绿=SSE 实时 / 橙=轮询降级 / 蓝=连接中 / 灰=离线。
- **刷新按钮**：手动重拉（通常不需要，会自动刷新）。
- **待拍板铃铛**：右上角橙色角标显示「有几件事等你拿主意」，点击进待拍板中心。

3.2 十三个视图（左侧导航）

视图	作用
 总览 Overview	跨所有项目的总账（项目数/任务数/已完工/待拍板）+ 每项目卡片（完成度环、状态分布、最近动态）。
 看板 Kanban	单项目按状态分泳道的看板墙。卡片显示进度、分支、PR、波次、待拍板角标。
 待拍板中心 ApprovalCenter	汇总所有项目「等人拍板」的问题，可在此 直接选项+确认拍板 。
 卡片抽屉 TaskDrawer	点任意卡片右侧滑出，展示该任务全字段 + 文档就地预览 + 内联拍板。
 活动流 ActivityFeed	时间线流水账。
 风险 RiskPanel	集中显示暂缓/阻塞/搁置的任务。
 波次 Waves	按 wave 分组，看施工批次。
 验收矩阵 AcceptanceMatrix	红黄绿灯表格：每任务的测试通过数、类型检查等质量信号。
 占用冲突 Collision	检测多个任务是否「抢同一分支/文件」，防并行施工打架。
 甘特 Gantt	时间条形图（echarts 懒加载）。
 依赖图 DependencyGraph	任务依赖/阻塞关系连线图。
 搜索筛选 SearchFilter	按名/号/状态筛选，导出快照。
 拍板历史 DecisionHistory &  待落地 ToLand	已拍板决策的历史（带落地徽标）+ 已拍板但尚未落地的清单（可复制「接单指令」派给新对话）。

3.3 拍板闭环（本产品的差异化功能）

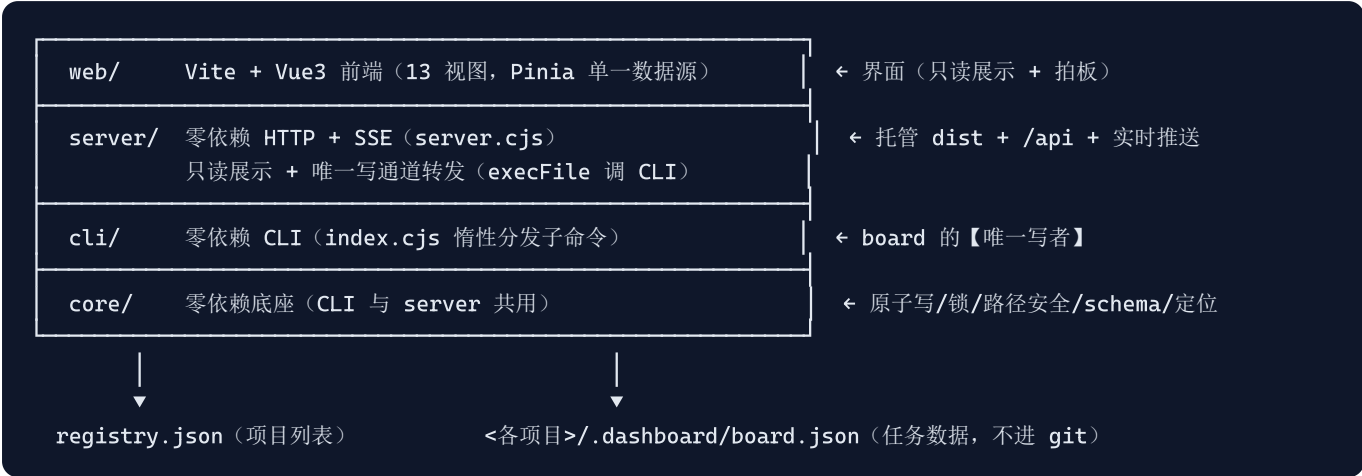
1. 施工方（人或 AI）遇到岔路口 → `cli pending` 登记一个决策（问题 + 若干选项 + 推荐项 + 推荐理由）。
2. 该任务状态可标为「待拍板」，铃铛角标 +1。
3. 人类在网页上选项 + 确认 → `POST /api/decide` → CLI 落库 → 界面几秒内 SSE 自动刷新，问题标上答案、从待拍板消失。
4. 决策可进一步标「已落地」（`mark-landed`），形成「问题→答案→落地」完整链路，避免拍了没人做。
5. **待落地**视图能把某任务所有已拍板未落地的决策打包成一份「任务书」，复制一句短指令粘到新的 Claude Code 对话即可让它接单施工。

3.4 实时刷新

- 后台每 1.5s 轮询各 board 文件的 mtime，一旦变化就通过 **SSE (Server-Sent Events)** 广播 `board:changed`，前端自动重拉，变化的卡片黄色闪一下。
- SSE 断开自动降级为 3s 轮询，恢复后自动切回 SSE。

4. 技术架构

4.1 四层 + 两份数据



- **零运行时依赖**: core / cli / server 只用 Node 内置模块 (http/fs/path/url/child_process/crypto)。前端 web 用 Vite+Vue+Pinia+vue-router+echarts, 但**只在开发/构建期**需要, 运行时只托管构建产物 web/dist。
- **Node 版本**: 开发/打包用 Node 24; 分发版内嵌 Node 运行时 (见 §9)。

4.2 core (底座, core/*.cjs)

文件	职责	治本点
atomicWrite.cjs	唯一写盘: 临时文件 + fsync + rename; Windows rename 覆盖失败时 unlink + 指数退避重试。	防写一半崩坏 / 防 Windows renameSync 覆盖报错。
lock.cjs	跨进程文件锁: O_EXCL 建锁 + 退避重试 + 陈旧锁抢占。	多进程/多对话并发写 board 串行化。
safePath.cjs	normalizeReal (realpath 规范化) / resolveInsideRoot (确保路径在根内)。用 path.relative 判断而非 startsWith。	防 ../ 和 junction 软链逃逸出项目根 (读文档 API 的安全底线)。
resolveProject.cjs	--project <id> → {mainRepo, board, lock, docsRoot}; 读 registry; 定义 DASHBOARD_HOME / REGISTRY_PATH。	显式定位, 不靠 cwd 猜 (worktree/junction 丛林里 cwd 会失灵)。
boardSchema.cjs	状态枚举、emoji、emptyBoard 工厂、手写零依赖 validate / assertValid。	写前校验拒坏数据; 统计 读时派生不落盘 。

4.3 数据流与写入纪律 (最重要的架构约束)

- **board 的唯一写者 = CLI**。一切写操作只能经 CLI → store.mutate(proj, mutator, activityEntry), 其内部保证: withLock → 锁内重读最新 → 就地改 → 刷 updatedAt → 追 activity → assertValid 校验 → atomicWrite。
- **server 绝不自写 board**。网页拍板走 POST /api/decide → server 用 execFile + 数组传参 (防命令注入) 调 cli decide → CLI 写。server 对 board 只读。
- **累加字段用 union-by-key 合并** (commitShas/prNumbers/gitBranch/activity), 绝不整对象覆盖, 防并发丢更新。
- **统计读时派生**: 进度%、状态计数、验收矩阵等一律读时算, 不存进 board (避免与真值漂移)。

4.4 数据落点与 DASHBOARD_HOME（重定位机制）

- `DASHBOARD_HOME` = 数据根，决定 `registry.json` 与 `snapshots/` 落哪。
- 默认（环境变量未设）：`~/.claude/dashboard`（Claude Code 集成布局）。
- 分发版：启动器把 `DASHBOARD_HOME` 指向**安装目录**，从而脱离 `~/.claude`、在任意社区机器上可写、卸载即净。
- 代码定位与 `DASHBOARD_HOME` 无关：`core/cli/server/web` 之间的 `require` 一律走 `__dirname` 相对路径。`DASHBOARD_HOME` 只决定「数据往哪读/写」，不决定「代码在哪」。这是分发版能同时满足「代码在安装目录、数据也在安装目录、又不改任何 Claude Code 用户行为」的关键。
- **board 本身不在 `DASHBOARD_HOME`**，而在**各项目主仓** `<mainRepo>/dashboard/board.json`。因此卸载看板不会丢各项目的任务数据（只丢项目列表 `registry` 与 `snapshots`）。

4.5 server 端点一览

方法 & 路径	作用
GET /api/health	探活： <code>{ok, service, port, pid, projects:[id], hooksInstalled, distBuilt, sseSubscribers}</code> 。
GET /api/projects	<code>registry</code> + 每项目 读时派生 摘要 (total/byStatus/progress/pendingCount/lastActivityTs)。
GET /api/board/:id	该项目 <code>board.json</code> 全量。
GET /api/doc?projectId=&path=	读项目内文档文本。path 必过 <code>safePath.resolveInsideRoot</code> ，逃逸/越界返回 403。
POST /api/decide/:pid/:taskId	转发 <code>execFile</code> 调 <code>cli decide</code> （数组传参防注入）。
POST /api/mark-landed/:pid/:taskId	标记决策已落地。
POST /api/dispatch* / GET inbox 相关	派单：生成「任务书」给新对话接手（Claude Code 集成，见 §8）。
GET /api/stream	SSE: mtime 轮询驱动 <code>board:changed</code> ；15s 心跳；断开清理订阅者； 禁用 fs.watch 。

4.6 单实例与端口

- 启动先探 `/api/health`，若发现同签名实例已在跑 → 复用 + 开浏览器，不重复起。
- 端口从 6060 起，被占则 6061...6079 自增。
- 绑定 `127.0.0.1`（仅本机访问，安全默认）。
- 环境变量：`DASHBOARD_PORT`（起始端口）/ `DASHBOARD_NO_OPEN=1`（不自动开浏览器）/ `DASHBOARD_POLL_MS`（轮询间隔）/ `DASHBOARD_HOME`（数据根）/ `DASHBOARD_REGISTRY`（测试隔离用，覆盖 `registry` 路径）。

5. 数据模型 (board.json 结构)

5.1 顶层结构

```
{
  "schemaVersion": "1.0",
  "project": {
    "id": "myapp",
    "name": "我的应用",
    "mainRepo": "F:\\myapp",
    "forbiddenZones": [],           // 全项目级「禁区」文件/目录
    "createdAt": "2026-07-02T10:00:00.000Z", // date-time
    "updatedAt": "2026-07-02T12:34:56.000Z"
  },
  "tasks": [ /* Task[] */ ],
  "activity": [ /* Activity[] */ ]
}
```

5.2 状态枚举 (STATUS, 共 10 种, 顺序即语义流转)

未开工 → 待开工 → 待拍板 → 已拍板 → 施工中 → 可复工 → 收官 → 已完工, 外加 暂缓、压轴。对应 emoji:  未开工 /  待开工 /  待拍板 /  已拍板 /  施工中 /  可复工 /  收官 /  已完工 /  暂缓 /  压轴。

注意「已拍板」与「已完工」都用 , 含义不同: 前者=决定定了, 后者=活干完了。

5.3 Task 字段

字段	类型	必填	说明
id	string	✓	任务号，正则 <code>^[A-Z0-9][A-Z0-9-]*\$</code> （必须大写开头，如 <code>P17</code> 、 <code>BUG-12</code> ）。
title	string	✓	标题。
status	string	✓	见 §5.2 枚举。
wave	int	✓	波次（批次），默认 0。
percent	int(0–100)		进度%。
description	string		一句话说明。
dates	object		<code>{design, start, done}</code> ，各为 <code>YYYY-MM-DD</code> 或 <code>null</code> 。
gitBranch / worktree / prNumbers / commitShas	array		git 派生字段（累加合并）。 <code>commitShas</code> 元素须匹配 <code>^[0-9a-f]{7,40}\$</code> 。
fileScope / forbiddenZones	array		涉及文件 / 禁区。
docs	array		相关文档路径。
deps	object		<code>{dependsOn[], blockedBy[], relatedTasks[]}</code> ，引用必须指向存在的 task id（校验强制）。
decisions	array		见 §5.4。

5.4 Decision 字段

字段	必填	说明
id	✓	决策号（如 <code>d1</code> ）。
question	✓	要拿主意的问题。
options	✓	至少一个选项（ <code>string[]</code> ）。
recommended	✓	推荐项。
answer		拍板答案。若非空且 <code>allowCustom</code> 不为真，则必须 $\in$ <code>options</code> （校验强制）。
allowCustom		允许自定义答案（绕开 <code>options</code> 约束）。
recommendReason		推荐理由。
decidedAt		拍板日期 <code>YYYY-MM-DD</code> 。
landed / landedAt / landedCommit		是否已落地 + 时间 + 落地 commit。

5.5 校验规则 (`validate`)

零依赖手写校验器，一次收集全部错误、带 JSON 路径。覆盖：必填/枚举/类型/日期正则/percent 范围/answer∈options/commit sha 格式/**引用完整性** (deps 与 activity.taskId 指向存在的 task、id 唯一)。**写前 `assertValid`**，坏数据直接拒绝落盘。

5.6 registry.json

```
{
  "schemaVersion": "1.0",
  "projects": {
    "<id>": { "name": "显示名", "mainRepo": "主仓绝对路径", "board": "board.json 绝对路径 (可选, 默认 <mainRe
  }
}
```

6. CLI 命令全集

调用形式：`node cli/index.cjs <命令> --project <id> [flags]` (分发版可用内嵌 node: `node-runtime\node.exe cli\index.cjs ...`)。全局 flag: `--author <身份>`、`--json` (机器可读输出)、`--registry <path>` (测试隔离)。

命令	用途
<code>register --id --name --root [--board]</code>	注册项目到 registry + 建空 board。
<code>import [--from <INDEX.md>] [--dry-run]</code>	从现有 markdown 任务台账 (INDEX/BOARD 表格) 批量回填任务骨架 (自动剥删除线、抽可靠字段)。
<code>backfill [--patch <json>]</code>	查缺语义字段 / 用补丁批量补。
<code>add <id> --title [--status --wave --desc]</code>	新建任务。
<code>claim <id> --branch [--scope]</code>	认领 → 施工中 (带防倒退状态机)。
<code>progress <id> --percent [--next --tests --typecheck]</code>	里程碑回写进度。
<code>sync-progress [--project]</code>	按当前 git 分支找施工中任务, 自动同步进度 (只进不退, 封顶 95)。
<code>pending <id> --q --opt... --rec</code>	登记待拍板问题 (问题/选项/推荐三件套)。
<code>decide <id> --did --answer [--promote]</code>	拍板 (填答案)。
<code>mark-landed <id> --did</code>	标记决策已落地。
<code>park <id> --reason / block <id> --by --reason</code>	暂缓 / 阻塞。
<code>done <id> [--pr --commit --collect]</code>	收官 / 完工。
<code>note --text [--task] / set <id> --field --value</code>	记活动流 / 通用兜底赋值。
<code>list [--status --wave] / show <id> --pending</code>	查询 / 待拍板中心。
<code>sync-from-git [--branch --n]</code>	从 git 自动派生 commit/pr/branch (hook 调用)。
<code>doctor [--fix]</code>	对账 git↔board + 自检 hook + 有边界的修复。
<code>render-index [--index <INDEX.md>] [--dry-run]</code>	从 board 生成 INDEX 状态段 (HTML 锚之间, 幂等)。
<code>snapshot [--out --stamp]</code>	导出 board 快照 (git 外备份)。
<code>inbox --project [--tid]</code>	「读看板接单」入口: 无 tid 列待落地任务, 给 tid 打印完整任务书。
<code>enroll / onboard</code>	一键接入全新项目 / 引导。
<code>hooks-install</code>	装 git hook + Claude Code hook (见 §8)。

7. 关键设计决策与「为什么」

这一节回答「它为什么这么设计」, 是理解本产品最有价值的部分。

0. **核心哲学：人不手动更新看板，任务由模型建链、经 CLI 全程写回。** 这是理解本产品的第一原理。使用范式是：人把任务派给模型（如 Claude Code）→ 模型接单、认领、施工、登记决策、收官，**每一步都由模型自己经 CLI 写回看板**，形成「认领→进度→待拍板→拍板→落地→完工」的完整任务链。人类全程**只做三件事**：看进度、在岔路口拍板、把新活派给模型。为什么不给人做「网页改状态/挪卡」：一旦让人手动维护任务状态，它就退化成一个「靠人勤快挪卡」的传统敏捷看板（Jira/Trello），而那正是本产品要超越的东西。**本产品的价值恰恰在于「人不用维护它，模型替人维护」**。所以「网页端不能新建任务/改状态」不是缺陷，是**刻意不做**——写入是模型的职责，人的写操作只保留「拍板」这一个决策动作。**推论**：本产品**依赖一个会经 CLI 写看板的施工方**（AI 模型 / 脚本 / hook）。没有这个施工方、纯靠人手填，它就用错了场景（见 \$10）。
1. **board 不进 git。** 为什么：board 是高频变动的运行时状态，多个对话/进程并发写。若进 git，多分支/多 worktree 会疯狂制造合并冲突和「撞号」。治本=board 落 `.dashboard/`、由 `.gitignore` / `.git/info/exclude` 忽略，历史留痕靠 `snapshot` 导出到 git 外。
2. **CLI 是唯一写者，server 只读 + 转发。** 为什么：单一写入通道才能保证「锁 + 校验 + 原子写 + 活动流」这套不变量在所有写路径上都成立。server 若也能写，就有两套写逻辑要同步，迟早漂移。网页拍板因此绕一圈走 `execFile` 调 CLI，宁可慢一点也不破坏「唯一写者」。
3. **命令注入防御：数组传参。** 为什么：server 转发到 CLI 用 `execFile(cli, [args ...])` 而非拼字符串 `exec`，用户输入的答案/项目名不会被 shell 解释，杜绝注入。
4. **路径安全用 realpath + path.relative，不用 startsWith。** 为什么：`startsWith` 判断「路径在根内」会被 `/root-evil` 或 junction 软链绕过。治本=先 `realpath` 解析真实路径，再 `path.relative` 看是否以 `..` 开头。读文档 API 的越界防御靠这个。
5. **实时用 mtime 轮询驱动 SSE，禁用 fs.watch。** 为什么：`fs.watch` 在 Windows/网络盘/多编辑器场景下事件不可靠（漏报、重复、跨平台语义不一）。治本=每 1.5s 读 board 文件 mtime，变了才广播。牺牲一点实时性换确定性与跨平台一致。
6. **统计读时派生、不落盘。** 为什么：进度%、状态计数若存进 board，就会和任务真值漂移（改了任务忘了改统计）。治本=永远从 tasks 现算。
7. **原子写 + 跨进程锁 + Windows 重试。** 为什么：多进程并发写 + Windows `renameSync` 覆盖已存在文件会抛错。治本=写临时文件→`fsync`→`rename`，失败则 `unlink` + 指数退避重试；写全程持文件锁串行化。
8. **单实例 + 端口自增 + 绑 127.0.0.1。** 为什么：用户会重复双击启动器；默认只该本机访问。治本=启动先探活复用、端口占用自增、只绑本地环回。
9. **三层数据新鲜度保险（弱→强→兜底）。** ①skill 指令让 AI 对话主动调 CLI（弱，靠记性）；②git `post-commit` hook 自动 `sync-from-git` 派生 commit/pr/branch（强，机械可靠）；③ `doctor` 对账抓漂移（兜底）。**git 派生字段与语义字段（decisions/wave/禁区）字段集不相交**，各有唯一权威、互不抢写。
10. **pre-commit 硬闸门（Claude Code 集成）。** 为什么：光靠文档说教，AI 对话可能「忘了」先认领任务再改代码。治本=在项目装 pre-commit hook，commit 前检查看板有没有当前分支的施工中任务，没有就**拒绝 commit** 并给补救命令（紧急放行 `DASHBOARD_SKIP_CLAIM_CHECK=1`）。这是代码层强制，凌驾于说教。
11. **派单用「短触发 + 读看板」而非「塞长 prompt 进命令行」。** 为什么：早期尝试 spawn 终端把上百行任务书塞进命令行，遇到转义炸裂 + 命令行长度限制 + 「开的是终端 claude 不是桌面 App」三重坑。治本=网页复制一句短指令（`... inbox --project x --tid Y`），粘到桌面 Claude Code 新对话，那对话自己跑 `inbox` 从看板拿完整任务书。
12. **DASHBOARD_HOME 环境变量重定位（分发版关键）。** 为什么：原设计数据锚死 `~/ .claude/dashboard`，对非 Claude Code 的社区用户既是错误品牌耦合、也可能不可写。治本=数据根用 `process.env.DASHBOARD_HOME`

覆盖、默认回退旧路径，向后兼容零影响。

8. Claude Code 深度集成（可选层）

这一层是本产品为 AI 驱动开发设计的差异化能力，但对不使用 Claude Code 的用户完全可选、不影响核心看板。

- **git hooks** (`hooks-install` 装到项目) : `post-commit` = 自动 `sync-from-git` + `render-index` ; `post-merge` = `sync-from-git` ; `pre-commit` = 未认领硬闸门。全部 `|| true` 兜底，绝不阻断正常 commit。
- **Claude Code hooks** (装到 `~/.claude/settings.json` 或项目 `.claude/settings.json`) : `Stop` = 每次对话结束跑 `doctor` 对账； `PostToolUse/Bash` = 检测 `git commit` 后自动 sync； `PostToolUse/ToDoWrite` = AI 更新待办清单时自动算完成比同步进度。
- **skill 集成**：配套的 `project-build-workflow` skill 在认领协议、看板同步、拍板话术等章节调用 CLI，让 AI 对话「自动知道规矩」。
- **派单**：网页把某任务的已拍板未落地决策打包成任务书，一句短指令派给新对话接手施工。

对社区非 Claude Code 用户：这些命令都在，但不主动生效；不装 hook、不用派单，看板照常作为「视图 + CLI + 拍板中心」工作。

9. 安装与分发（社区版）

9.1 形态

- **Windows NSIS 安装器**，约 **22MB** (LZMA solid 压缩)，**内嵌 Node 运行时**，安装后约 89MB。
- 用户**无需预装 Node.js**、**无需联网**、**无需命令行**即可启动。
- 安装到 `%LOCALAPPDATA%\Programs\ProjectDashboard` (**per-user, 不需要管理员权限**)，创建桌面 + 开始菜单快捷方式。

9.2 安装目录布局

<安装目录>\	(= DASHBOARD_HOME)
core\ cli\ server\ web\dist\	← 应用代码 + 前端构建产物
node-runtime\node.exe	← 内嵌 Node 运行时
registry.json	← 项目列表（初始为空）
启动看板.bat	← 主启动器（设 DASHBOARD_HOME + 起 server）
添加项目.bat	← 向导式 register 包装
使用手册.md / 开始使用.txt	← 文档
NOTICE-第三方声明.txt	← 内嵌 Node.js 的 MIT 许可声明
uninstall.exe	

9.3 打包流水线（如何再生产一个安装器）

在装有 Node 的开发机上：

```
node packaging/build-installer.cjs [--version 1.0.0]
```

脚本做 5 件事：①搭建 staging（拷 core/cli/server + web/dist + package.json + 手册，剔除 test/node_modules）②生成启动器/助手/说明 + 干净空 registry ③拷入 `process.execPath` 当运行时 ④生成 `installer.nsi` ⑤调 `makensis` 编译。产物落 `packaging/dist/`。 - **makensis 来源**：优先用 electron-builder 缓存里的 NSIS 3；没有则提示到 nsis.sourceforge.io 装（脚本不联网下载）。 - **铁律**：脚本只读源码、只写 `packaging/` 下的 staging/dist，绝不改被打包的源文件；shipped registry 永远为空（不带打包机的私人项目路径）。

9.4 未签名说明

安装器**未做代码签名**，Windows SmartScreen 可能提示。用户需点「更多信息 → 仍要运行」。代码签名证书是收费项，列入打磨清单。

10. 已知不足与边界（务必如实告知）

这些不是 bug，是当前产品的真实边界。回答「这工具能不能.....」类问题时以此为准。

1. 需要一个「会经 CLI 写看板」的施工方，否则看板不会自己更新（这是**适用场景边界**，非缺陷）。 - 本产品按 \$7 核心哲学设计：写入由 AI/模型经 CLI 完成，人不手动改状态。因此它**依赖模型/脚本/hook 来驱动**。 - 若你完全不接 AI、也不用 CLI，只想用鼠标手动挪卡改状态——本产品**不适合**你，请用传统看板（Trello/Jira/思源/Obsidian Kanban 等）。 - 分发版提供 `添加项目.bat` 做**一次性项目登记**（这是设置、不是日常维护）；日常任务的建链/更新是模型经 CLI 的职责。
2. **单机、无鉴权、绑 127.0.0.1**。无多人协作、无账号权限、无远程访问（默认）。局域网/远程分享需要自行反代且**当前无任何鉴权**，不建议直接暴露。
3. **拍板不可在网页撤销**。网页拍板一锤定音、当场落库。拍错了需用 CLI 改。
4. **仅 Windows 分发**。源码支持 macOS/Linux（有 `dashboard.sh`），但社区分发版目前只有 Windows NSIS。
5. **未签名安装器**（见 §9.4）。
6. **无自定义品牌图标**。当前快捷方式/卸载项复用 node.exe 图标（避免误用其它产品图标）。
7. **前端主 JS chunk 约 1MB**（含 echarts）。已做 echarts 懒加载（进甘特/依赖图才载），但主包仍偏大。
8. **依赖新鲜度部分靠约定**。git 派生字段全自动可靠；但 decisions/wave/禁区等语义字段仍需施工方主动登记（有 hook 缓解，非 100% 自动）。
9. **Claude Code 集成的路径假设**。派单短触发命令里的 CLI 路径按 `~/ .claude/dashboard` 布局生成；分发版装到别处时，若同时用 Claude Code 派单，该路径与实际安装位置不一致（核心看板不受影响）。属集成层边角，列入打磨。
10. **无自动更新**。升级需重新下载安装器覆盖安装。

11. 典型使用流程

11.1 首次启动（社区用户）

1. 下载安装器 → 双击 →（SmartScreen 提示则「仍要运行」） → 选安装位置 → 完成，勾选「立即启动」。

2. 浏览器自动打开 `http://127.0.0.1:6060/`，此时项目列表为空。
3. 双击「添加项目.bat」，按提示填：项目代号（英文）、显示名、项目文件夹路径 → 回车。
4. 回网页刷新，即见该项目（初始 0 任务）。
5. 用 CLI 给项目加任务：`node-runtime\node.exe cli\index.cjs add TASK1 --project <代号> --title "标题"`。

11.2 AI 驱动开发 (Claude Code 用户)

1. `enroll / register` 接入项目 + `import` 从现有任务台账回填 + `hooks-install` 装 hook。
2. AI 对话认领任务（`claim`）→ 施工 → `commit` (hook 自动 sync 进度/commit) → `done`。
3. 遇岔路口 → `pending` 登记决策 → 人类网页拍板 → `mark-landed` → 完成闭环。

11.3 日常

双击桌面「项目管理看板」→ 看总览/看板/待拍板 → 有橙色角标就去拍板 → 关掉黑窗口即停。

12. 扩展与集成点

- **接新项目**：`register --id --name --root` 一行；有 INDEX/BOARD 表格可 `import` 自动抽任务。
- **对接外部**：所有数据可经 `GET /api/board/:id` 拿到全量 JSON；`GET /api/projects` 拿派生摘要。只读集成很容易。
- **加视图**：前端加页面需同时改 `router/index.ts` 与 `components/SideNav.vue` 的 NAV 数组（只改路由会导致「页面存在但侧栏看不到」——这是一个真实踩过的坑）。
- **测试**：`node --test`（零依赖 node:test），当前 62 用例覆盖 core / 命令层 / 解析层 / server / 集成。

13. 快速事实卡（给模型的速查）

- 类型：本地单机、多项目、读多写少的项目治理面板。
- 技术：Node 零依赖后端（core/cli/server）+ Vue3/Vite 前端；数据 = registry.json + 各项目 board.json。
- 写入：CLI 是唯一写者；**写入由 AI/模型经 CLI 完成、人不手动改状态**（核心哲学，见 §7.0，刻意不做网页改状态以区别于传统看板）；网页给人的写操作只有「拍板」；board 不进 git。
- 实时：mtime 轮询 → SSE 广播。
- 安全：绑 127.0.0.1；无鉴权；读文档有 realpath 越界防御；命令注入用数组传参防御。
- 分发：Windows NSIS 安装器 ~22MB，内嵌 Node，per-user 安装，离线可用，未签名。
- 最大边界：依赖模型/CLI 驱动（人不手动改状态，是设计不是缺陷）；不接 AI/CLI 的纯手工场景不适用；单机无协作。
- 差异化：拍板闭环（问题→答案→落地）+ Claude Code 深度集成（hook/派单/pre-commit 闸门）。
- 状态枚举：未开工/待开工/待拍板/已拍板/施工中/可复工/收官/已完工/暂缓/压轴。
- 任务 id：必须匹配 `^[A-Z0-9][A-Z0-9-]*$`（大写开头）。